

ChainSight 供应链仿真系统 算法优化测试报告

1. 测试概述

1.1 测试目的

本报告对 ChainSight 供应链仿真系统进行了全面的三版本对比测试，验证代码重构后的功能正确性和性能提升效果。

1.2 测试版本

版本	说明	代码位置
Dev	原始开发版本（基准版本）	ChainSight_Dev/
Src	重构本地运行版本	src/
DB	重构数据库集成版本	src/ + --use-db

1.3 测试环境

- 操作系统: Windows 11
- Python版本: 3.x
- 数据库: PostgreSQL 5432 + DuckDB (内存模式)
- 测试数据: BC_S5.xlsx 配置文件
- 仿真周期: 5天 (2025-10-06 至 2025-10-10)
- 随机种子: 42 (确保可重复性)
- 测试时间: 2026-01-31

1.4 测试输出目录

版本	输出路径
Dev	ChainSight_Dev/BC_S5/run_20260129_204512/
Src	outputs/BC_S5/run_20260131_192824/
DB	outputs/db_BC_S5_20260131_202336/

2. 性能对比分析

2.1 总体运行时间

版本	开始时间	结束时间	总运行时间	平均每天
Dev	20:45:12	20:54:29	557秒 (9.28分钟)	111.4秒
Src	19:28:24	19:32:10	226秒 (3.77分钟)	45.25秒
DB	20:23:38	20:24:32	54.52秒	10.90秒

2.2 性能提升比例

对比项	加速倍数	时间减少百分比
Src vs Dev	2.46x	59.4%
DB vs Dev	10.22x	90.2%
DB vs Src	4.15x	75.9%

2.3 各模块平均耗时对比（秒/天）

模块	Dev	Src	DB	Src加速	DB加速
M1 订单生成	~7.4	~1.3	~1.2	5.69x	6.17x
M3 流量计算	~21.8	~3.2	~1.5	6.81x	14.53x
M5 部署规划	~59.5	~24.7	~6.5	2.41x	9.15x

2.4 关键模块每日详细耗时

Module 1 订单生成（秒）

日期	Dev	Src	DB
Day 1 (10-06)	6.66	1.27	1.19
Day 2 (10-07)	12.74	1.23	1.18
Day 3 (10-08)	5.94	1.37	1.17
Day 4 (10-09)	5.85	1.21	1.19
Day 5 (10-10)	5.95	1.39	1.25

Module 3 净需求计算 (秒)

日期	Dev	Src	DB
Day 1 (10-06)	22.84	3.40	1.48
Day 2 (10-07)	21.83	3.23	1.50
Day 3 (10-08)	22.51	3.42	1.60
Day 4 (10-09)	23.33	3.42	1.55
Day 5 (10-10)	24.44	3.63	1.70

Module 5 部署规划 (秒)

日期	Dev	Src	DB
Day 1 (10-06)	70.09	22.87	6.15
Day 2 (10-07)	54.58	23.90	6.39
Day 3 (10-08)	55.39	24.62	6.55
Day 4 (10-09)	60.15	25.81	6.70
Day 5 (10-10)	57.41	26.23	6.77

2.5 DB版本资源使用

项目	值
DuckDB 内存限制	4GB
线程数	14
仿真核心运行	54.52秒
数据库写入开销	已包含在总时间内

3. 数据一致性验证

3.1 总体验证结果

验证项	Dev	Src	DB	一致性
Module1 OrderLog	8,250	8,250	8,250	☑ 100%
Module1 ShipmentLog	1,410	1,410	1,410	☑ 100%
Module1 CutLog	1,410	1,410	1,410	☑ 100%
Module3 NetDemand	2,615	2,615	2,615	☑ 100%
Module4 ProductionPlan	64	64	64	☑ 100%
Module5 DeploymentPlan	59,044	59,044	59,044	☑ 100%
Module6 DeliveryPlan	637	637	637	☑ 100%

3.2 业务指标验证

指标	Dev	Src	DB	一致性
总订单数	8,250	8,250	8,250	☑
发运数量	1,410	1,410	1,410	☑
削减数量	1,410	1,410	1,410	☑
净需求记录	2,615	2,615	2,615	☑
有效部署计划	1,489	1,489	1,489	☑
交付计划	637	637	637	☑

3.3 每日部署计划详细对比

日期	Dev	Src	DB
2025-10-06	64	64	64
2025-10-07	102	102	102
2025-10-08	415	415	415
2025-10-09	431	431	431
2025-10-10	477	477	477
总计	1,489	1,489	1,489

3.4 每日交付计划详细对比

日期	Dev	Src	DB
2025-10-06	0	0	0
2025-10-07	103	103	103
2025-10-08	52	52	52
2025-10-09	391	391	391
2025-10-10	91	91	91
总计	637	637	637

4. Bug修复记录

4.1 本次测试修复的问题

在测试过程中，发现并修复了以下数据一致性问题：

Bug 1: Module1 OrderLog 累积订单字段不一致

问题描述: DB版本写入数据库时使用了 `orders_df` 而非累积订单 `all_orders_for_next_day`

修复文件: `pgsql_db/module_data_writer.py` (约第659行)

```
# 修复前
'all_orders_for_next_day': ('module1_output_orderlog', 'orders_df'),

# 修复后
'all_orders_for_next_day': ('module1_output_orderlog', 'all_orders_for_next_day'),
```

Bug 2: Module1 集成模块累积订单逻辑错误

问题描述: 主集成模块使用了错误的字段名

修复文件: `src/core/main_integration.py` (约第2258行)

```
# 修复前
accumulated_orders = day_results.get('orders_df', pd.DataFrame())

# 修复后
accumulated_orders = day_results.get('all_orders_for_next_day', pd.DataFrame())
```

Bug 3: Module4 净需求处理逻辑不完整

问题描述: 净需求应只包含下游需求 (Layer 0) , 且数量需取绝对值

修复文件: `src/core/main_integration.py` (约第797-812行)

```
# 修复后添加
net_demand_df = net_demand_df[net_demand_df['location_layer'] == 0].copy()
if 'quantity' in net_demand_df.columns:
    net_demand_df['quantity'] = net_demand_df['quantity'].abs()
```

Bug 4: Module5 内存模式数据加载问题 (关键修复)

问题描述: Module5 在内存模式下未能正确读取累积订单数据, 导致 AO (Auto-Order) 需求丢失

修复文件: `src/modules/deployment_planning/data_loader.py` (约第342行)

```
# 修复前
orders_df = module1_result.get('orders_df', pd.DataFrame())

# 修复后
orders_df = module1_result.get('all_orders_for_next_day',
module1_result.get('orders_df', pd.DataFrame()))
```

影响: 此Bug导致 DB 版本的 Module5 部署计划数量与 Dev 基准不一致。修复后三版本数据完全一致。

Bug 5: 数据库写入前未清空历史数据

问题描述: 多次运行会导致数据累积

修复文件: `pgsql_db/module_data_writer.py`

- 添加 `truncate_output_tables()` 方法
- 修改 `write_module_results_from_dict()` 在写入前自动清空表

5. 技术优化详解

5.1 代码架构优化

优化项	优化前 (Dev)	优化后 (Src/DB)
代码结构	单文件混合	模块化分层
数据处理	逐行操作	向量化批处理
内存管理	多次复制	原地操作
I/O操作	每天写文件	批量/内存缓存
数据存储	纯Excel文件	DuckDB内存 + PostgreSQL

5.2 模块级优化

Module 1 订单生成 (6.17x 加速)

- 使用 NumPy 向量化操作替代 Python 循环
- AO/Normal 消耗计算完全向量化
- 批量生成随机数，避免逐行调用

Module 3 净需求计算 (14.53x 加速)

- DuckDB 批量计算替代逐节点 Python 计算
- 按层级批量处理 (Layer 2 → 1 → 0)
- 单次 DuckDB 查询处理数百节点

Module 5 部署规划 (9.15x 加速)

- 预构建索引加速查找
- 需求收集算法优化
- 分配算法向量化处理

5.3 DB版本特有优化

技术	说明
DuckDB 内存模式	列式存储，OLAP 查询优化
批量写入	使用 <code>copy_from</code> 高效导入
内存数据传递	模块间直接传递 DataFrame
自动表清理	写入前自动 TRUNCATE

6. 测试结论

6.1 功能验证结论

☒ 重构版本(Src/DB)与原始版本(Dev)输出100%一致

验证覆盖：

- Module1: 订单、发运、削减日志
- Module3: 净需求计算结果
- Module4: 生产计划
- Module5: 部署计划
- Module6: 交付计划
- 所有业务指标数值

6.2 性能提升结论

版本	相对Dev加速	推荐场景
Src	2.46x	常规仿真运行、调试开发
DB	10.22x	大规模数据分析、生产环境

6.3 优化效果总结

指标	优化前(Dev)	Src版本	DB版本	最佳提升
单次仿真时间	9.28分钟	3.77分钟	54.52秒	90.2%
M1耗时/天	7.4秒	1.3秒	1.2秒	83.8%
M3耗时/天	21.8秒	3.2秒	1.5秒	93.1%
M5耗时/天	59.5秒	24.7秒	6.5秒	89.1%

6.4 建议

- 1. **生产环境:** 推荐使用 DB 版本，性能提升显著
- 2. **开发调试:** Src 版本足够使用，无需数据库依赖
- 3. **数据分析:** DB 版本支持 SQL 查询，便于数据分析

7. 附录

7.1 测试日志文件

版本	日志路径
Dev	ChainSight_Dev/BC_S5/run_20260129_204512/simulation_log_20260129_204512.txt
Src	outputs/BC_S5/run_20260131_192824/simulation_log_20260131_192824.txt
DB	outputs/db_BC_S5_20260131_202336/simulation_log_20260131_202336.txt

7.2 测试命令

```
# Dev版本
cd ChainSight_Dev
python run.py --case BC_S5.xlsx --start-date 2025-10-06 --end-date 2025-10-10

# Src版本（本地运行）
python run.py --config config/BC_S5.xlsx --start-date 2025-10-06 --end-date 2025-10-10

# DB版本（数据库集成）
python run.py --config BC_S5 --start-date 2025-10-06 --end-date 2025-10-10 --use-db
```

7.3 数据库连接

```
import psycopg
conn = psycopg.connect('host=localhost port=5432 dbname=test_db user=postgres
password=123456')
```

7.4 数据对比脚本

```
# 对比 DB vs Dev 基准
python compare_db_vs_dev.py

# 清空数据库输出表
python truncate_tables.py
```

报告生成时间: 2026-01-31

测试工程师: chenxianyue002@chinasofti.com

版本: v2.0